Integración Lista de precios ICX (CRM → B2C Commerce) · Resumen de la sesión

Prueba técnica Developer Salesforce · Imagine CX · Entrega: miércoles 28 de octubre de 2026 (fin del día) Fuente: `HU_Prueba_Tecnica_Developer_Salesforce.pdf` y `Mapa de la integración ICX.md` (ambos en esta carpeta). Resumen escrito el 25 de octubre de 2026 y actualizado el 26 de octubre de 2026.

---

1. Qué piden

Un job Apex en el sandbox CRM, ejecutable bajo demanda, que:

1. Lea los productos activos de la lista de precios **Lista de precios ICX** (`Pricebook2`) y sus precios (`PricebookEntry`).
2. Por cada producto revise si existe en Commerce; si no existe lo cree en el catálogo; si existe no lo duplique.
3. Asigne o actualice el precio en un price book de Commerce.
4. Al terminar, los productos se vean en Commerce con sus precios.

Además, configuración **manual** en Business Manager: asignar categoría (Huevos / Embutidos), dejar el producto visible y con precio, reindexar y verificar en la tienda.

Datos mínimos y reglas de negocio

- Producto: código único (ProductCode) y nombre. Precio: valor de la lista ICX y moneda.
- El ProductCode identifica al producto en ambos ambientes; correr el job varias veces no duplica.
- Si cambia un precio en el CRM, la siguiente ejecución lo actualiza en Commerce.
- La integración **no** asigna categorías.

Restricciones técnicas

- Todo nativo en el CRM (Apex y, opcionalmente, Flows). Commerce se actualiza por sus APIs.
- Sin middleware ni herramientas externas (MuleSoft, Zapier, n8n...).
- Credenciales y endpoints fuera del código (configuración segura de Salesforce).
- Clases de prueba con cobertura mínima 75 % y callouts simulados.
- El job debe poder ejecutarse manualmente en la sustentación; dejarlo programado suma puntos.

Fuera de alcance

Categorías automáticas, imágenes, inventario, descripciones extendidas, automatizar el reindex, sincronización Commerce → CRM, eliminación de productos. Manejo de errores: no obligatorio, pero preguntarán cómo se resolvería en producción y suma si se implementa.

Criterios de aceptación (se demuestran en vivo)

CA	Escenario	Resultado esperado
CA1	Carga inicial con 4 productos	Los 4 quedan en Commerce con precio
CA2	Agregan un producto nuevo a la lista durante la sesión	Al correr el job se crea en Commerce con precio
CA3	Se vuelve a correr el job	Sin duplicados; precios iguales a los del CRM
CA4	Asignación manual de categoría en BM	El producto se ve en la tienda con nombre y precio

Sustentación (60 min)

1) Mostrar carga inicial en BM. 2) Mostrar los 4 productos en la tienda por categoría. 3) Agregan producto nuevo. 4) Correr el job y mostrarlo en Commerce. 5) Asignar categoría y mostrar tienda. 6) Explicar arquitectura, código, mapeo y configuración de Commerce. 7) Preguntas, incluido manejo de errores.

Entregables: integración funcionando entre los dos sandboxes y repositorio (GitHub u otro) con metadata, código y pruebas. Dudas: WhatsApp a Laura Garzón.

---

2. Hallazgos verificados sobre las APIs de Commerce

- **OCAPI está deprecada desde abril de 2026.** Salesforce indica que toda implementación nueva debe usar SCAPI. OCAPI sigue disponible dos años más solo con parches de seguridad.
- **La Data API de OCAPI no tiene recurso de price books.** Sus recursos son: Catalogs, Categories, Products, ProductInventoryRecords, Promotions, Campaigns, Sites, Jobs, etc. Por tanto, **no** se pueden escribir precios con OCAPI Data API. (Corrige el "plan B" del mapa: el único plan B real es la importación de price book por archivo XML desde Business Manager, o disparar ese job de importación por API.)
- **SCAPI Admin sí tiene las dos APIs necesarias** (especificaciones oficiales `products-oas-v1` y `price-books-oas-v1`):

Paso	API	Método y ruta	Scope
Verificar existencia	Products	`GET https://{shortCode}.api.commercecloud.salesforce.com/product/products/v1/organizations/{organizationId}/products/{productId}` → 200 existe / 404 no existe	`sfcc.products`
Crear producto	Products	`PUT .../products/{productId}` → 201 creado, 200 sobrescrito	`sfcc.products.rw`
Actualizar nombre	Products	`PATCH .../products/{productId}`	`sfcc.products.rw`

Paso	API	Método y ruta	Scope
Crear/actualizar precio	Price Books	`PUT https://{shortCode}.api.commercecloud.salesforce.com/product/price-books/v1/organizations/{organizationId}/price-books/{priceBookId}/price-book-entries/{productId}` → 201 creado / 200 reemplazado (upsert idempotente)	`sfcc.price-books.rw`
Crear price book	Price Books	`PUT .../price-books/{priceBookId}` (requiere `id` y `currencyCode`; la moneda es inmutable)	`sfcc.price-books.rw`
Asignar price book al sitio	Price Books	`PUT .../price-books/{priceBookId}/site-assignments/{siteId}`	`sfcc.price-books.rw`

Cuerpos mínimos:

```
// PUT producto (campo requerido: id)
{
  "id": "HUE-AA-30",
  "owningCatalogId": "<catalogo-maestro>",
  "name": { "default": "Huevos AA x30" },
  "onlineFlag": { "<SiteId>": true },
  "searchable": { "<SiteId>": true }
}
```

```
// PUT entrada de price book (campo requerido: productId)
{
  "productId": "HUE-AA-30",
  "priceTables": [
    { "tiers": [
      { "type": "amount", "quantity": 1,
        "amount": { "currencyMnemonic": "COP", "value": 18500 } }
    ] }
  ]
}
```

Notas:

- `onlineFlag` y `searchable` son **mapas por sitio** (`{"RefArch": true}`), no booleanos simples. `online` es calculado (solo lectura).
- El PUT de producto sobre uno existente **sobrescribe** el documento. Por eso: GET primero, PUT solo si 404, PATCH del nombre si cambió.
- `priceTables` se reemplaza completo en cada PUT/PATCH; perfecto para CA3.
- Los errores de SCAPI siguen RFC 7807; conviene enviar un header `correlation-id` y guardarlo en el log.
- Timeout de Admin API: 60 s (504 si se excede).

Autenticación (Account Manager, client credentials)

- Endpoint de token: `https://account.demandware.com/dwsso/oauth2/access_token`
- `grant_type=client_credentials`

- `scope=SALESFORCE_COMMERCE_API:{tenant_id} sfcc.products.rw sfcc.price-books.rw` (el tenant_id es `{realm}_{instancia}`, p. ej. `zzxy_010`).
- El token es un JWT Bearer y dura 30 minutos; reutilizarlo dentro de la ejecución.
- El API client en Account Manager debe tener el rol **Salesforce Commerce API** con filtro de tenant al sandbox, los scopes permitidos, método de autenticación `client_secret_post` y formato de token JWT. Un cliente creado solo para OCAPI no sirve tal cual.
- `organizationId` tiene la forma `f_ecom_{realm}_{instancia}`; `shortCode` es un código de 8 caracteres. Ambos están en BM: Administration › Site Development › Salesforce Commerce API Settings.

---

3. Solución propuesta

Configuración segura en el CRM (nada en código)

- **External Credential** OAuth 2.0, flujo *Client Credentials with Client Secret*, URL del proveedor = endpoint de token, scope como arriba, principal con client id y secret. Plan B si el flujo estándar no negocia bien con Account Manager: pedir el token desde Apex con un Named Credential que guarde id y secret como principal.
- **Named Credential** `SFCC_Admin_API` con URL `https://{shortCode}.api.commercecloud.salesforce.com`.
- **Permission set** que otorgue acceso al principal al usuario que corre el job (incluido el usuario del job programado).
- **Custom Metadata Type** para lo no secreto: organizationId, siteId, owningCatalogId, priceBookId de Commerce, nombre del Pricebook2 del CRM, moneda por defecto.

Diseño Apex

Clase	Rol
`ICX_PriceSyncBatch`	`Database.Batchable<SObject>`, `Database.AllowsCallouts`, `Database.Stateful`. Scope ≈ 25 (cada producto usa hasta 3 callouts; límite 100 por transacción).
`ICX_PriceSyncScheduler`	`Schedulable` que encola el batch (punto extra).
`CommerceApiService` / cliente SCAPI	getProduct, createProduct, updateProductName, upsertPrice, createPriceBook, assignPriceBookToSite; manejo de token y errores.
Invocable / Quick Action	Botón o Screen Flow "Sincronizar con Commerce" para lanzar el job en vivo.
`Commerce_Sync_Log__c`	Una fila por producto y ejecución (estado, HTTP, mensaje, correlation-id) y una fila SUMMARY.

SOQL de origen:

```
SELECT Id, UnitPrice, Product2.ProductCode, Product2.Name
FROM PricebookEntry
WHERE Pricebook2.Name = :nombreLista AND IsActive = true AND Product2.IsActive = true
```

Si la org tiene multimoneda, agregar `CurrencyIsoCode`; si no, usar `UserInfo.getDefaultCurrency()` o la moneda del Custom Metadata (consultar con SOQL dinámico para que compile en ambos casos).

Flujo por ejecución (coincide con el diagrama del mapa):

1. Leer entradas activas de la lista ICX.
2. Verificar/crear el price book de la moneda y su asignación al sitio.
3. Por producto: GET → si 404, PUT (crear en catálogo maestro, online y searchable) → si 200 y cambió el nombre, PATCH.
4. PUT del precio (upsert) siempre.
5. Registrar fila en el log; en error 5xx reintentar y, si persiste, registrar y continuar.
6. Fila SUMMARY con exitosos y errores.

Mapeo de campos

CRM	B2C Commerce	Regla
`Product2.ProductCode`	`product.id` y `productId` del precio	Llave única; idempotencia
`Product2.Name`	`product.name.default`	Se crea con el producto; se actualiza si cambia
`PricebookEntry.UnitPrice`	`priceTables[0].tiers[0].amount.value`, `quantity: 1`	Upsert en cada ejecución
Moneda (entrada u org)	`currencyMnemonic` y moneda del price book `icx-list-prices-{moneda}`	Debe coincidir con la moneda habilitada en el sitio
Configuración	`owningCatalogId`, `onlineFlag`, `searchable`	Categorías no se tocan

Pruebas

`HttpCalloutMock` que enrute por método y URL: 404 en GET de producto nuevo, 200 en existente, 201/200 en PUT, 500 para error. Pruebas del batch (`Test.startTest/stopTest`), scheduler, servicio e invocable cubriendo CA1, CA2, CA3 y errores. Objetivo ≥ 75 %.

Configuración manual en Business Manager

1. Identificar catálogo maestro (donde crea el job) y catálogo de la tienda (donde están las categorías).
2. Price book ICX en la moneda del CRM, online y asignado al sitio; confirmar que la moneda esté permitida en el sitio.
3. Tras el job: asignar productos a categorías Huevos / Embutidos; confirmar online y searchable para el sitio.
4. Desmarcar "Show Orderable Products Only" en las preferencias de búsqueda o dar inventario perpetuo (sin esto, productos sin inventario pueden no listarse).
5. Reindexar (Merchant Tools › Search › Search Indexes), invalidar caché y verificar la tienda.

Respuesta preparada: "¿Qué pasa si Commerce no responde a mitad de la carga?"

Cada producto es una unidad independiente: un fallo no detiene a los demás. El error queda en el log, el batch continúa y al final se envía un resumen (email o Platform Event). Como todo es idempotente, la siguiente ejecución repara lo pendiente. Mejoras para producción: reintentos con backoff para 429/5xx, timeout explícito, renovación del token ante 401, campo de última sincronización en `PricebookEntry` para procesar solo cambios, alertas sobre el log, y correlation-id para rastrear en Log Center.

---

4. Estado actual y pendientes

Estado del proyecto (`prueba-tecnica-anibal`)

- Proyecto SFDX recién creado (scaffold estándar): `force-app/main/default/classes` y demás carpetas están **vacías** por ahora.
- El código que el mapa marca como "hecho" se generó en una sesión web de Claude Code y **no está en este equipo**; hay que traerlo (push al repo desde la sesión web o copiar los archivos a este proyecto).
- En `docs/`: el PDF de la prueba, el mapa de la integración y este resumen.

Checklist

Accesos (bloqueantes)

- □ Client ID y Secret del API client de Commerce con rol Salesforce Commerce API y scopes `sfcc.products.rw`, `sfcc.price-books.rw` (pedir a Laura o crear en Account Manager).
- □ Short Code y Organization Id (BM › Administration › Site Development › Salesforce Commerce API Settings).
- □ Probar con curl/Postman un token y un `GET /price-books` antes de escribir Apex. Si la API de Price Books no responde, escalar de inmediato.
- □ Repositorio en GitHub.

Setup CRM

- □ External Credential + Named Credential + permission set.
- □ Custom Metadata con catálogo maestro, sitio, price book y lista ICX reales.
- □ Traer el código, desplegar, correr pruebas (≥ 75 %).
- □ Verificar que los 4 productos tengan ProductCode y anotar la moneda (¿org multimoneda?).

Commerce y tienda

- □ Moneda del sitio = moneda del CRM.
- □ Correr el job y validar CA1 en BM.
- □ Categorías, flags, orderable-only / inventario, reindex, caché, verificar tienda (CA4).

Sustentación

- □ Ensayar CA2 y CA3 (agregar producto, cambiar precio, correr dos veces).
- □ Dejar el job programado.

- □ Preparar explicación de arquitectura, mapeo, configuración y errores.

Plan sugerido

Día	Foco
Sáb 25	Accesos, probar token y un GET, credenciales en el CRM, traer el código al repo
Dom 26	Desplegar, correr carga inicial, validar en BM
Lun 27	Pruebas y cobertura, log, configuración manual en BM, tienda, README, push
Mar 28	Ensayo completo de la demo y job programado

Riesgos

- Credenciales que no sean de SCAPI Admin o sin permisos en Account Manager para agregar scopes (bloqueante).
- API de Price Books no habilitada en el sandbox (plan B: importación XML).
- Productos sin ProductCode o con caracteres no válidos para id de Commerce (máx. 100).
- Diferencia de moneda entre CRM y sitio.
- Límite de callouts si se sube el tamaño del lote.

---

5. Fuentes consultadas

- Why Use SCAPI (deprecación de OCAPI): https://developer.salesforce.com/docs/commerce/commerce-api/guide/why-use-scapi.html
- OCAPI OAuth 2.0: https://developer.salesforce.com/docs/commerce/b2c-commerce/references/b2c-commerce-ocapi/oauth.html
- Authorization for SCAPI Admin APIs: https://developer.salesforce.com/docs/commerce/commerce-api/guide/authorization-for-admin-apis.html
- Authorization Scopes Catalog: https://developer.salesforce.com/docs/commerce/commerce-api/guide/auth-z-scope-catalog.html
- Products resource, OCAPI Data API 23.2 (espejo): https://sfcclearning.com/infocenter/OCAPI/current/data/Resources/Products.php
- Especificaciones SCAPI Products y Price Books (`schemas/scapi/product/products/v1.json` y `.../price-books/v1.json`): https://github.com/SalesforceCommerceCloud/b2c-developer-tooling
- Named Credentials Developer Guide: https://developer.salesforce.com/docs/platform/named-credentials/guide/nc-oauth-dev-guide.html

---

6. Avance del 26 de octubre: API client de Commerce creado

- Organización en Account Manager: **Italcol SA** (el realm `blsb` pertenece a esta org; el sandbox de la prueba es `blsb-001`).
- API client creado: **ICX CRM PriceSync (prueba tecnica)**, Client ID `a6fb362f-d807-4377-aef1-5d1392cf56b4`, Confidential, habilitado.

- Rol: Salesforce Commerce API con filtro de tenant `blsb-001.dx.commercecloud.salesforce.com` (`blsb_001`).
- Allowed scopes: `sfcc.products.rw`, `sfcc.price-books.rw`, `sfcc.catalogs`. Token Endpoint Auth Method: `client_secret_post`.
- Prueba de token: HTTP 200, `expires_in` 1800, scopes devueltos `SALESFORCE_COMMERCE_API:blsb_001 sfcc.products.rw sfcc.price-books.rw`.
- El secret está en `credenciales-commerce-NO-SUBIR.txt`, fuera del proyecto SFDX. No se sube a Git.
- Datos SCAPI: Organization ID `f_ecom_blsb_001`. Short Code `eeluxar4` (Active).
- Capturas del proceso en `docs/capturas-account-manager/`.

Siguiente paso: ver sección 7.

---

7. Validación del 26 de octubre: estado real de los dos sandboxes

7.1 Commerce `blsb-001` (Business Manager)

Capturas: `capturas-account-manager/06-bm-commerce-api-settings.png` y `07-bm-manage-sites-sin-sitios.png`.

Dato	Valor
Short Code	`eeluxar4` (Active)
Organization ID	`f_ecom_blsb_001`
Sitios de tienda	**Ninguno** ("this sales organization has no Storefront sites")
Catálogos	**0** (`GET /product/catalogs/v1/.../catalogs` → `total: 0`)
Código desplegado	Solo `version1` (vacía), creada el 26/10/2026 21:22 UTC
Importaciones disponibles	"Storefront Reference Architecture Demo Sites" y "SiteGenesis Demo Site" en Site Import & Export
WebDAV Client Permissions	Vacío (configurable por nosotros si hubiera código que desplegar)

Pruebas de API con el token del cliente nuevo (lectura):

- `GET .../catalogs` → 200, lista vacía. Autenticación y scopes correctos.
- `GET .../products/NO-EXISTE-123` → 404 `ProductNotFound`. Es la señal que usará el job para decidir crear.
- `GET .../price-books` → 404 `Site Not Found`: el listado necesita un sitio; sin sitios no responde.
- La API de Catálogos de SCAPI **no permite crear catálogos** (solo GET, PATCH y búsquedas). El catálogo maestro se crea a mano en BM o llega con la importación demo.

Conclusión: el sandbox fue aprovisionado el mismo 26 de octubre y está vacío. El PDF dice que el sandbox "contiene Business Manager y la tienda en línea" y el CA4 se valida en la tienda. Hoy eso no existe.

7.2 CRM `enterprise-efficiency-9228--mkt` (alias sf CLI `pruebaTecnicaAnibal`)

Dato	Valor
Org	Italcol SA, Enterprise Edition, sandbox, API 68
Usuario	`cotesanibal3@gmail.com.mkt`, perfil Administrador del sistema
Multimoneda	No. Moneda por defecto **COP**
Lista de precios ICX	Existe (`01sWD000003C0H3YAK`), activa
Entradas ICX	**2, no 4**: CARNES (código 1020, 35.400) y POLLOS (código 2020, 23.400). POLLOS tiene el **producto inactivo**
Integración existente	Nada: sin Named/External Credentials, sin clases ICX, sin jobs programados
Proyecto local	`force-app` contiene 88 clases y otra metadata traída de la org (no es entregable). Se creó `icx-integration` como package directory separado para el desarrollo de la prueba

7.3 ¿Qué podemos hacer nosotros y qué hay que pedir?

Necesidad	¿Nosotros?	Detalle
Sitio, catálogo y price books en Commerce	**Sí**	Importar "Storefront Reference Architecture Demo Sites" desde BM, o crear a mano un catálogo maestro y un sitio. Cubre CA1, CA2 y CA3 (se validan en BM)
Tienda en línea (storefront)	**No**	Requiere desplegar el código SFRA. El repositorio de SFRA en GitHub es privado (acceso solo para partners/clientes autorizados; la cuenta de GitHub del equipo recibe 404). Sin storefront no hay CA4
4 productos en la lista ICX	**Sí**	Somos administradores en el CRM. Recomendación: no tocar POLLOS (puede ser intencional para validar el filtro de activos) y agregar productos propios activos con ProductCode
Moneda	**Sí**	COP en CRM y en el price book de Commerce; habilitar COP en el sitio cuando exista

7.4 Mensaje para Laura (único punto que no depende de nosotros)

> Entré al sandbox blsb-001 y no tiene sitios, catálogos ni storefront desplegado, aunque la prueba indica que contiene la tienda en línea y el CA4 se valida ahí. Del lado del CRM y de las APIs ya tengo todo conectado y puedo importar los datos demo en Business Manager, pero para la tienda necesito que ustedes desplieguen el storefront o me den acceso al código SFRA. ¿Cómo prefieren resolverlo?

7.5 Conceptos para la sustentación

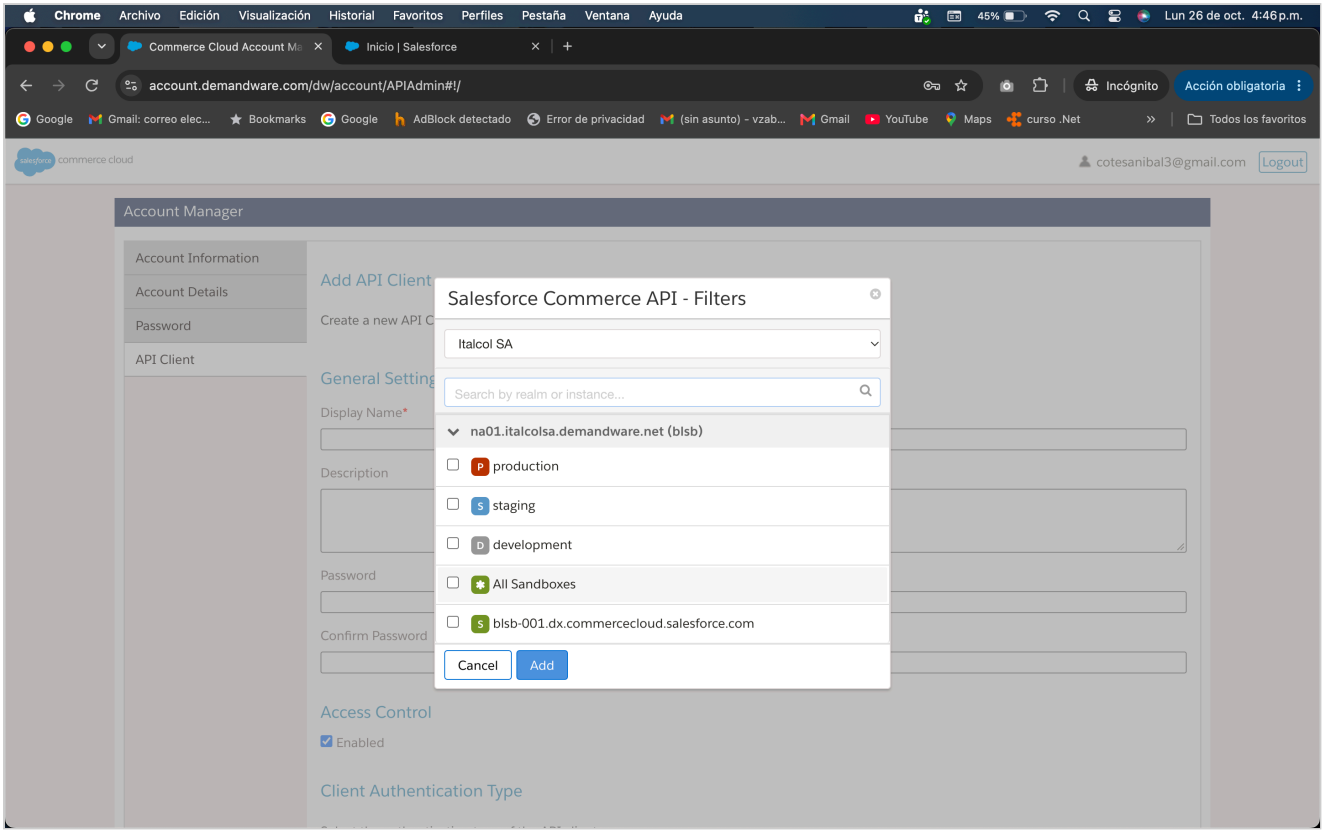
- **Storefront:** la tienda en línea que ve el comprador. No viene con el sandbox; es código que se despliega en la instancia.
- **SFRA (Storefront Reference Architecture):** la tienda de referencia de Salesforce, un conjunto de cartridges con tienda completa. Casi todo proyecto parte de ahí.
- **Sitio:** configuración de una tienda concreta (idioma, monedas, catálogo de tienda, price books). `blsb-001` no tiene ninguno.

- **Catálogo maestro vs. catálogo de tienda:** en el maestro nacen los productos (ahí crea el job); el de tienda tiene las categorías que ve el comprador y es donde se asigna la categoría a mano.
- **Orden de armado:** sitio + catálogos + price books (datos) → código del storefront (SFRA) → job crea productos y precios → asignación de categoría y reindex → la tienda los muestra.

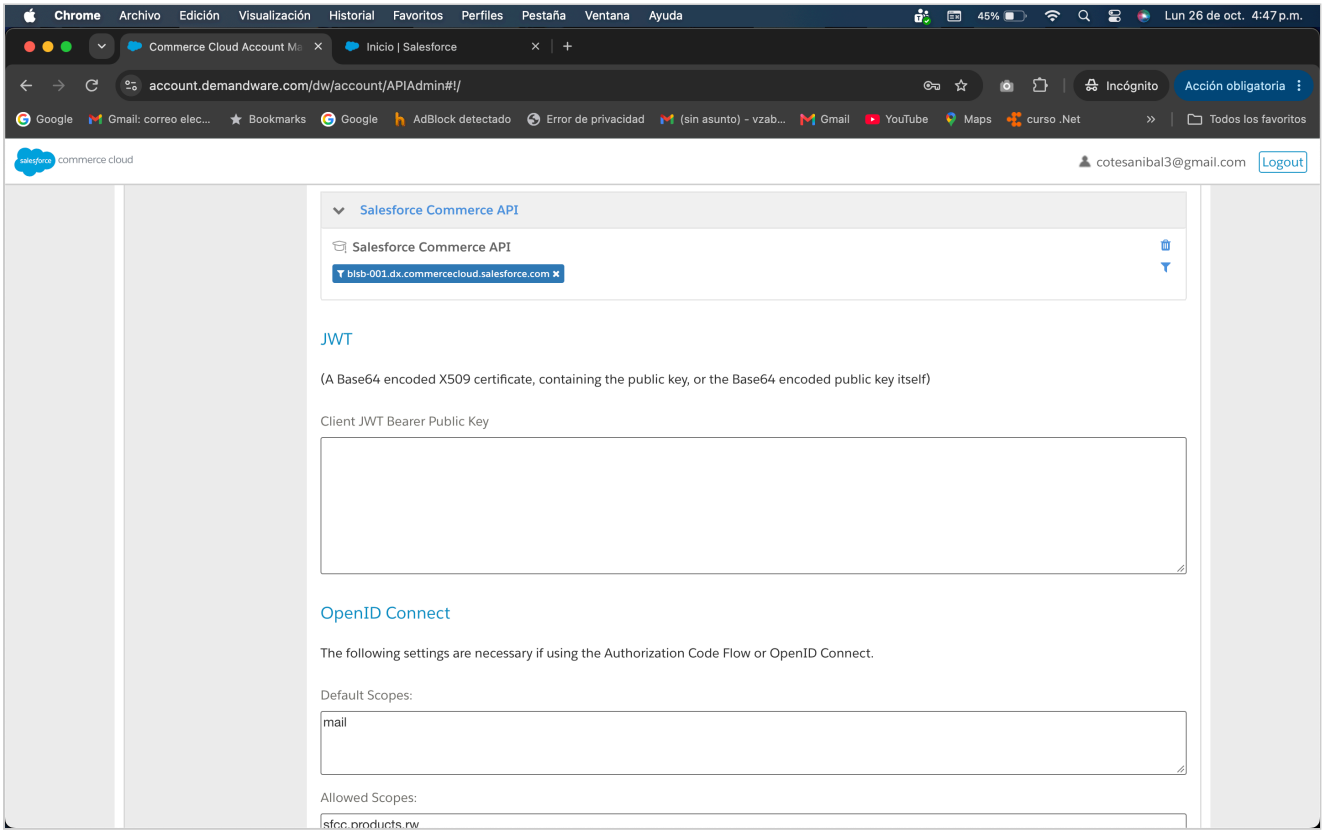
7.6 Estado del proyecto local (nada desplegado)

- `icx-integration/main/default/` contiene: objeto `Commerce_Sync_Log__c` (13 campos), Custom Metadata `Commerce_Sync_Setting__mdt` (10 campos) con registro `Default`, External Credential `SFCC_Account_Manager`, Named Credential `SFCC_Admin_API`, permission set `ICX_Commerce_Sync` y tab del log.
- Las clases Apex **no** se han escrito. Repositorio git inicializado en local (rama `main`), sin commits y sin remoto.
- **Nada se ha desplegado** a ninguno de los dos sandboxes.

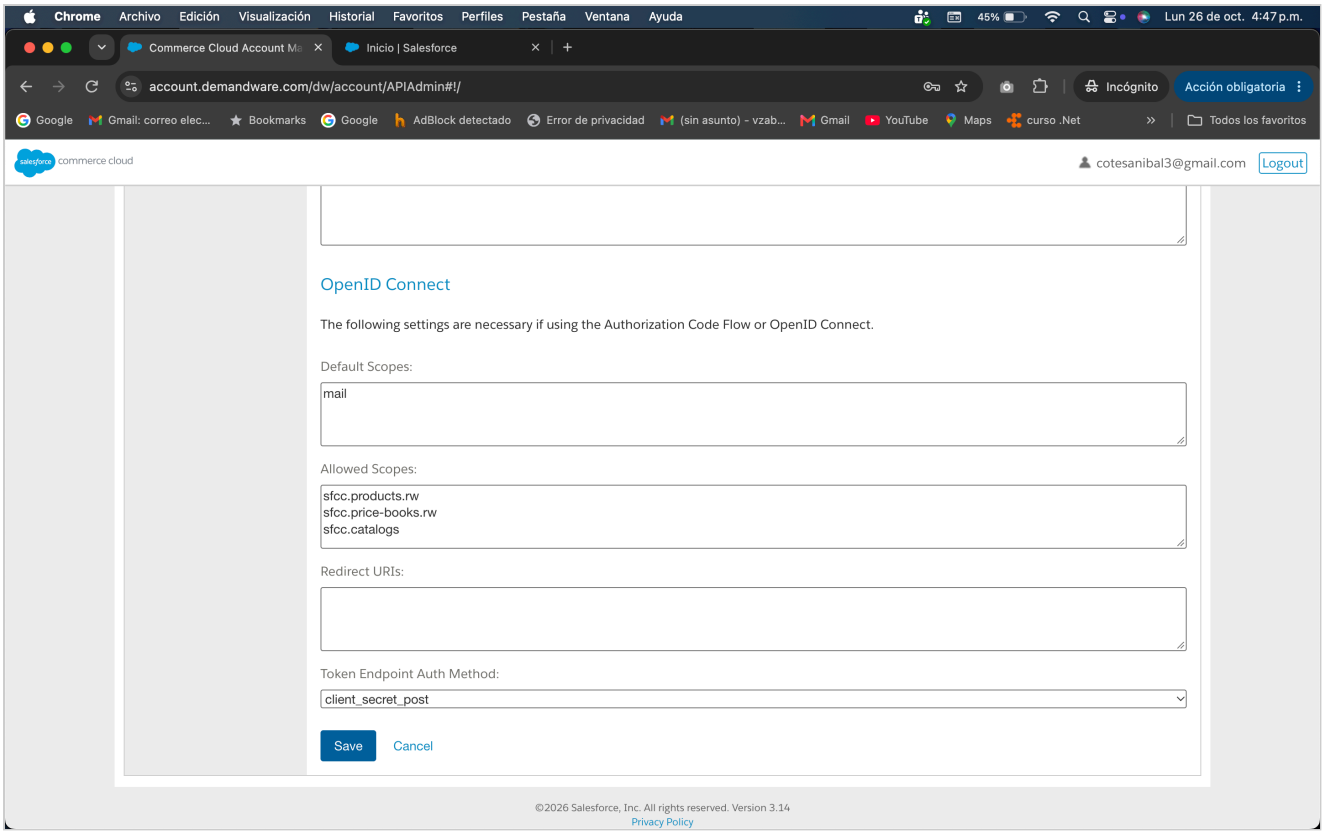
Anexo: capturas del proceso



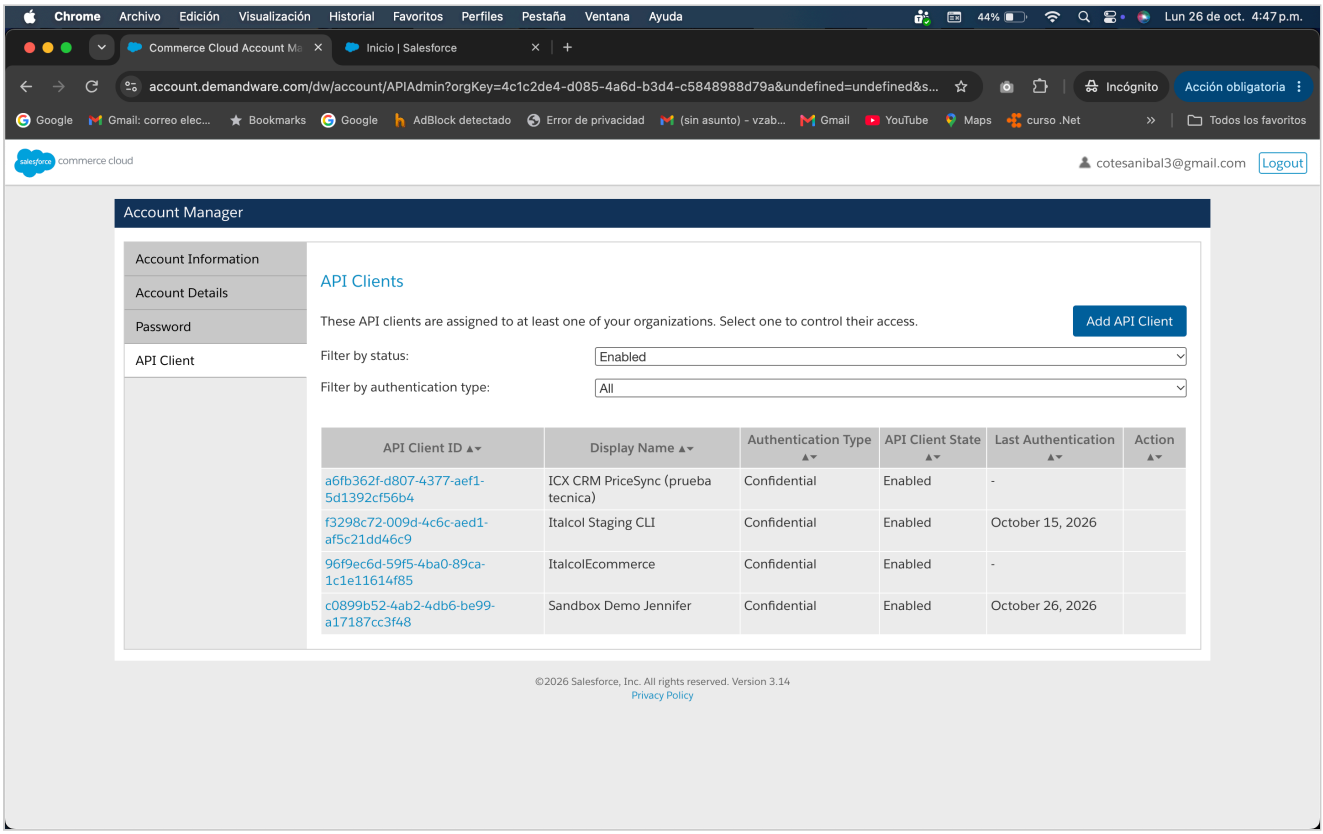
01-tenants-disponibles-italcol-blsb.png



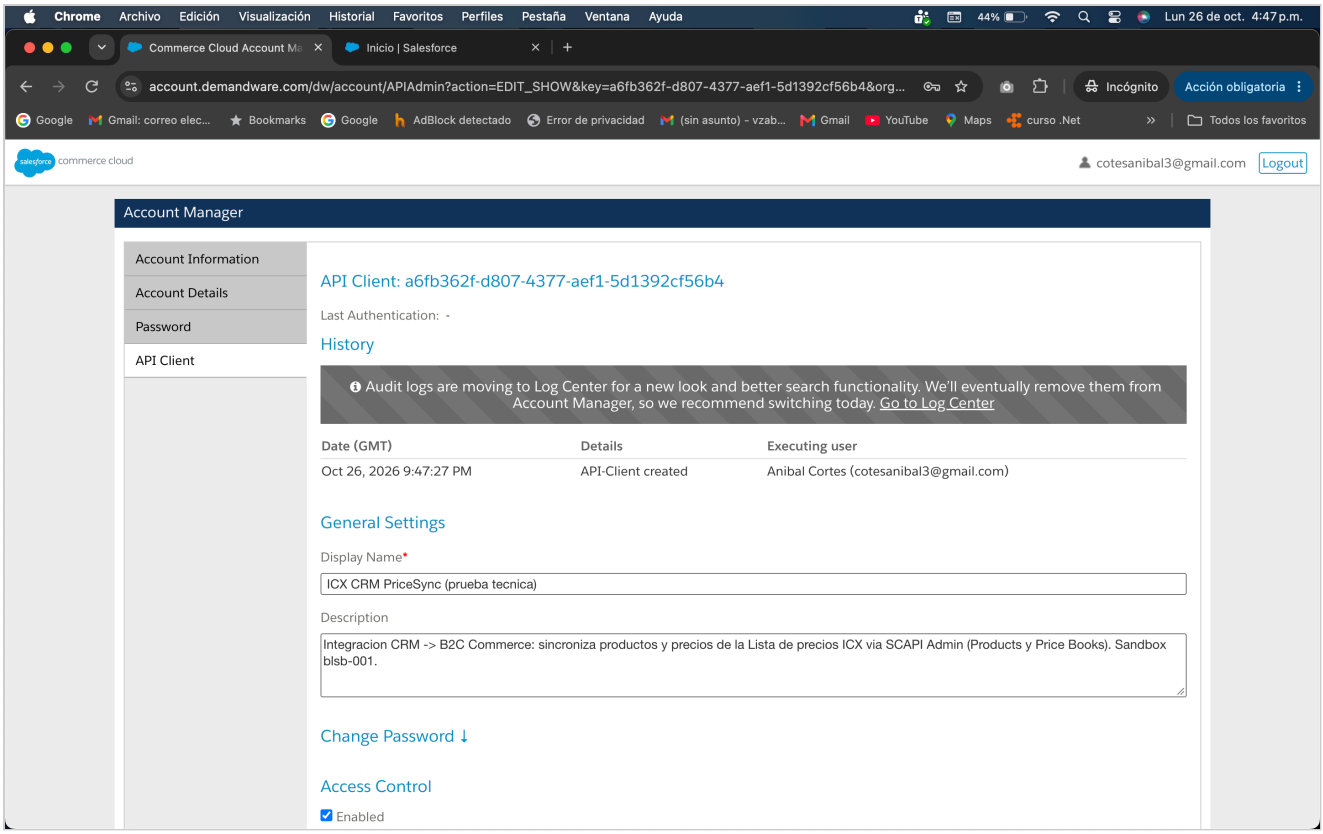
02-formulario-roles-tenant.png



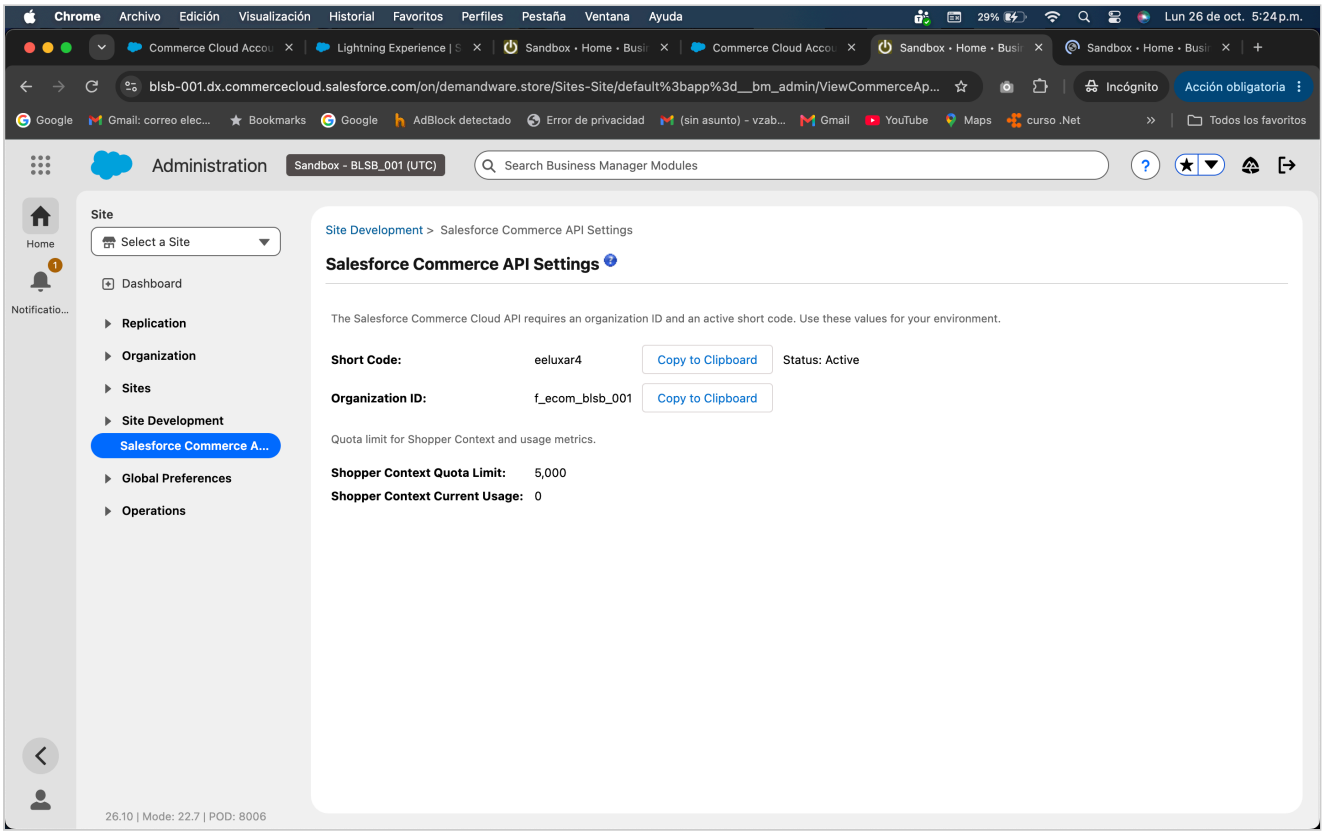
03-formulario-scopes-auth.png



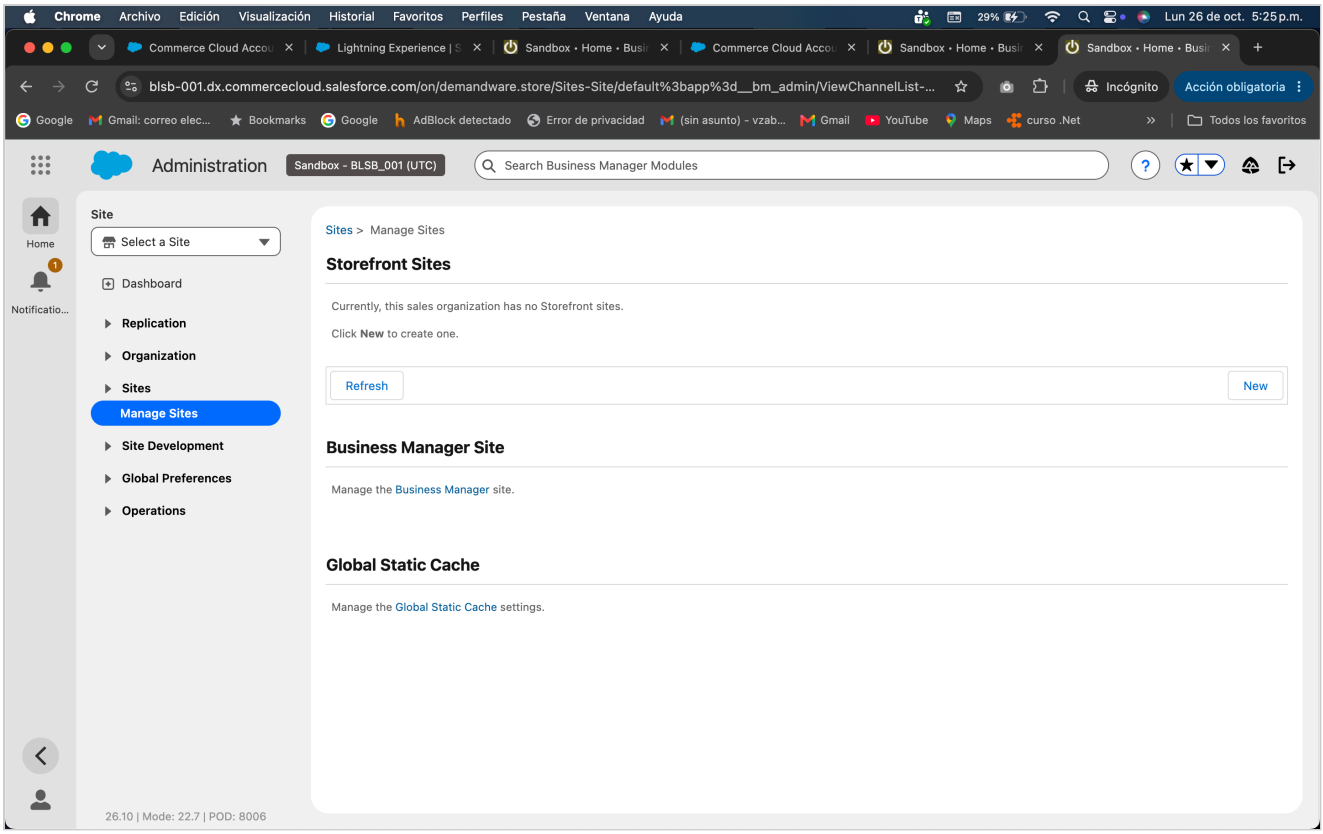
04-cliente-creado.png



05-detalle-cliente.png



06-bm-commerce-api-settings.png



07-bm-manage-sites-sin-sitios.png